



Mini Project Report - 10

Master of Computer Application – GENERATIVE AI
Semester – I

Sub: FRONT-END FRAMEWORKS AND TECHNOLOGIES

Topic: DYNAMIC TO – DO LIST APPLICATION

By

Name: R KUSHMA

Reg no.: PROV/ASAC/MCA/7/26/031

Faculty Name: VEERA RAGHAV K

Faculty Signature: _____

**Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106**

August 2026

INDEX

FIG NO	NAME OF TABLE	PG NO
1	INTRODUCTION	2
2	INPUT CODE	3 – 6
3	OUTPUT CODE	7
4	CONCLUSION	8

INTRODUCTION

The **Dynamic To-Do List** is a web-based task management application developed using **HTML, CSS, and JavaScript**. The main purpose of this project is to help users add, manage, update, complete, and delete their daily tasks in a simple and organized manner. HTML is used to create the basic structure of the webpage, including the heading, input box, buttons, and task display area.

CSS is used to improve the appearance of the application by adding background colours, spacing, fonts, buttons, sections, and completed-task styling.

JavaScript is used to make the To-Do List dynamic and interactive. Users can enter a task in the input box and click the **Add Task** button to add it to the task list. Each task contains options such as **Complete, Edit, and Delete**.

When a task is completed, it is displayed with a line-through effect and a grey colour. The application also provides filtering options such as **All, Pending, and Completed**, allowing users to view tasks according to their current status. The **Clear Completed** button removes all tasks that have already been completed.

The project also uses **JavaScript arrays and objects** to store task information. Each task contains a task name and a completed status. The `displayTasks()` function dynamically updates the task list on the webpage. The project uses **DOM manipulation** through functions such as `getElementById()`, `createElement()`, and `appendChild()` to display and modify tasks.

It also uses **localStorage**, which allows the task information to remain available even after refreshing the webpage. Overall, the project demonstrates important frontend concepts such as HTML structure, CSS styling, JavaScript functions, events, DOM manipulation, arrays, objects, conditional statements, and browser local storage.

INPUT CODE

```
<html>
<head>
  <title>Dynamic To-Do List</title>
  <style>
    body {
      font-family: Arial;
      margin: 0;
      background-color: #f3e8f0;
    }

    header {
      background-color: #7b2d5f;
      color: white;
      text-align: center;
      padding: 20px;
    }

    section {
      background-color: white;
      margin: 20px;
      padding: 20px;
    }

    h2 {
      color: #7b2d5f;
    }

    input {
      padding: 10px;
      width: 60%;
    }

    button {
      background-color: #7b2d5f;
      color: white;
      padding: 10px;
      border: none;
      margin: 5px;
      cursor: pointer;
    }

    button:hover {
      background-color: #5e2048;
    }

    .task {
```

```

        background-color: #f5e6f0;
        padding: 10px;
        margin: 10px 0;
    }

    .completed {
        text-decoration: line-through;
        color: gray;
    }
</style>
</head>
<body>
    <header>
        <h1>DYNAMIC TO-DO LIST</h1>
    </header>

    <section>
        <h2>ADD TASK</h2>
        <input type="text" id="taskInput" placeholder="Enter your task">
        <button onclick="addTask()">Add Task</button>
    </section>

    <section>
        <h2>MY TASKS</h2>
        <button onclick="showAll()">All</button>
        <button onclick="showPending()">Pending</button>
        <button onclick="showCompleted()">Completed</button>
        <button onclick="clearCompleted()">Clear Completed</button>
        <p id="count"></p>
        <div id="taskList"></div>
    </section>

    <script>
        var tasks = JSON.parse(localStorage.getItem("tasks")) || [];
        var filter = "all";
        function displayTasks() {

            var list = document.getElementById("taskList");
            list.innerHTML = "";
            for (var i = 0; i < tasks.length; i++) {
                if (filter == "pending" && tasks[i].completed == true) {
                    continue;
                }
                if (filter == "completed" && tasks[i].completed == false) {
                    continue;
                }
                var div = document.createElement("div");
                div.className = "task";
                if (tasks[i].completed == true) {
                    div.className = "task completed";
                }
            }
        }
    </script>

```

```

    }
    div.innerHTML =
        tasks[i].name +
        " <button onclick='completeTask(" + i + ")'>Complete</button>" +
        " <button onclick='editTask(" + i + ")'>Edit</button>" +
        " <button onclick='deleteTask(" + i + ")'>Delete</button>";

    list.appendChild(div);
}
document.getElementById("count").innerHTML =
    "Total Tasks: " + tasks.length;
localStorage.setItem("tasks", JSON.stringify(tasks));
}

function addTask() {
    var task = document.getElementById("taskInput").value;
    if (task == "") {
        alert("Please enter a task");
    }
    else {
        tasks.push({
            name: task,
            completed: false
        });

        document.getElementById("taskInput").value = "";
        displayTasks();
    }
}

function completeTask(index) {
    tasks[index].completed = true;
    displayTasks();
}

function editTask(index) {
    var newTask = prompt(
        "Enter new task",
        tasks[index].name
    );

    if (newTask != null && newTask != "") {
        tasks[index].name = newTask;
        displayTasks();
    }
}

function deleteTask(index) {
    tasks.splice(index, 1);
    displayTasks();
}

```

```

    }

    function showAll() {
        filter = "all";
        displayTasks();
    }

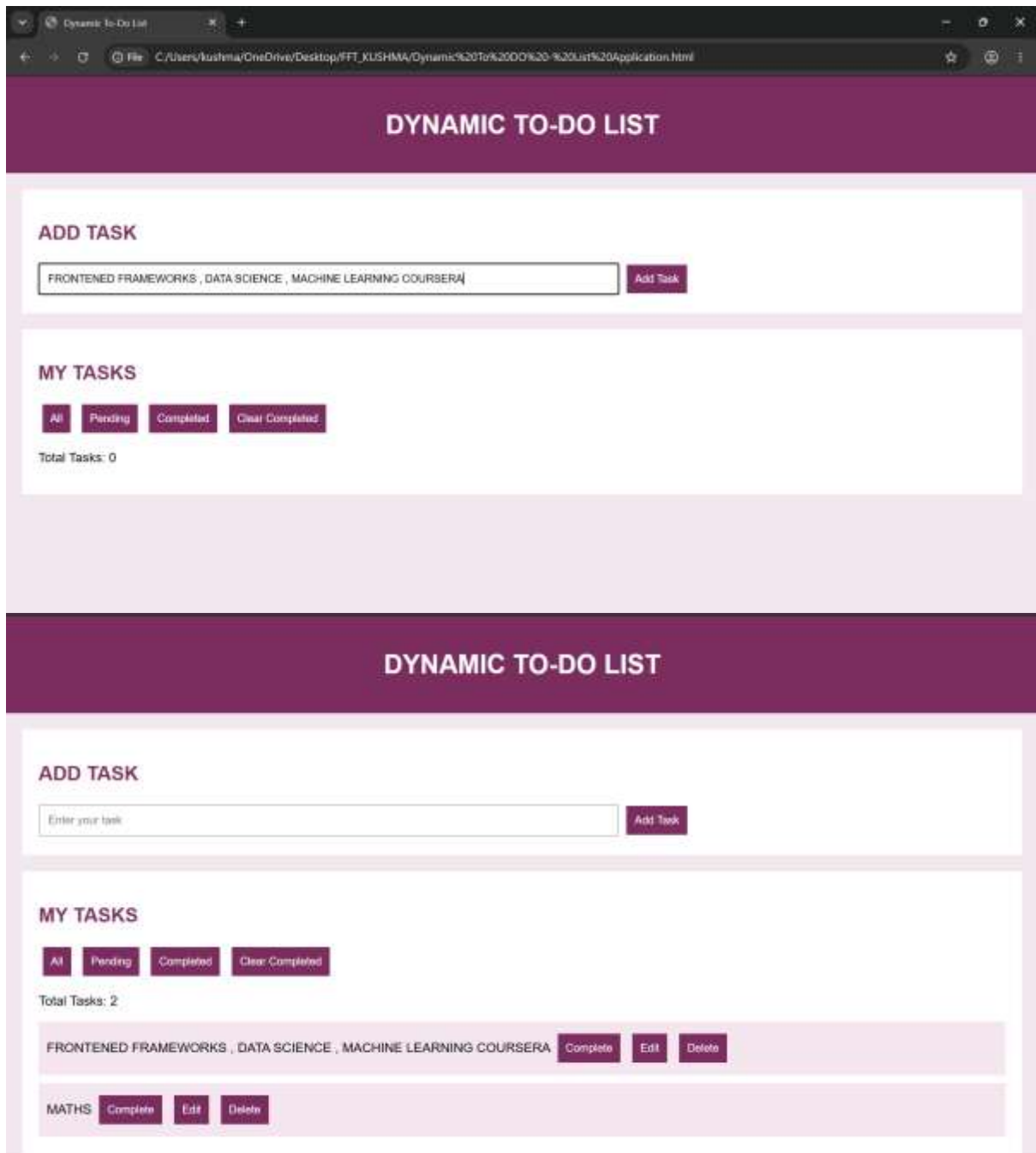
    function showPending() {
        filter = "pending";
        displayTasks();
    }

    function showCompleted() {
        filter = "completed";
        displayTasks();
    }

    function clearCompleted() {
        var newTasks = [];
        for (var i = 0; i < tasks.length; i++) {
            if (tasks[i].completed == false) {
                newTasks.push(tasks[i]);
            }
        }
        tasks = newTasks;
        displayTasks();
    }
    displayTasks();
</script>
</body>
</html>

```

OUTPUT CODE



CONCLUSION

The **Dynamic To-Do List** successfully demonstrates how HTML, CSS, and JavaScript can be combined to develop an interactive and useful web application. The project provides basic task-management features such as adding new tasks, completing tasks, editing task names, deleting tasks, filtering tasks, and clearing completed tasks. These features make the application simple to use and demonstrate how JavaScript can be used to create dynamic changes on a webpage without reloading it.

Through this project, important concepts of **Frontend Frameworks and Technologies** have been practically implemented. HTML provides the structure of the application, CSS improves its visual presentation, and JavaScript provides the functionality and interaction. The use of DOM manipulation helps in dynamically creating and updating task elements, while localStorage provides a way to retain the task data in the browser. The project also gives practical understanding of JavaScript arrays, objects, loops, functions, conditions, and event handling.

In conclusion, the Dynamic To-Do List is a simple but effective frontend project that demonstrates the fundamental concepts required to build interactive web applications. It can be further improved by adding features such as task due dates, priorities, search functionality, notifications, dark mode, and user login. Thus, the project provides a good practical foundation for understanding how frontend technologies work together to solve a real-world task-management problem.